



Yenepoya (Deemed To Be University)

(A constituent unit of Yenepoya Deemed to be University)

Deralakatte, Mangaluru – 575018, Karnataka, India

FLIGHT DELAY PREDICTION SYSTEM

PROJECT FINAL REPORT

BACHELOR OF COMPUTER APPLICATIONS

Big Data Analytics, Cloud Computing, Cyber Security with IBM

SUBMITTED BY

AP Adhnan – 22BDACC002

Christo Johny Parakkadan – 22BDACC063

AP Adhil – 22DBACC003

Aslam C – 22BDACC043

Guided By

Mr. Sumit Kumar Shukla

TABLE OF CONTENTS

SL NO	TITLE	PAGE NO
	Executive Summary	1
1	Background	2
1.1	Aim	2
1.2	Technologies	2
1.3	Hardware Architecture	3
1.4	Software Architecture	3
2	System	4
2.1	Requirements	4
2.1.1	Functional Requirements	5
2.1.2	User Requirements	5
2.1.3	Environmental Requirements	5
2.2	Design and Architecture	6
2.3	Implementation	6
2.4	Testing	6
2.4.1	Test Plan Objectives	7
2.4.2	Data Entry	7
2.4.3	Security	8
2.4.4	Test Strategy	8
2.4.5	System Test	8
2.4.6	Performance Test	9
2.4.7	Security Test	9
2.4.8	Basic Test	10
2.4.9	Stress and Volume Test	10
2.4.10	Recovery Test	10
2.4.11	Documentation Test	11
2.4.12	User Acceptance Testing	11
2.4.13	System Testing	12
2.5	Graphical User Interface (GUI) Layout	12
2.6	Customer Testing	12
2.7	Evaluation	13
2.7.1	Performance	13
2.7.2	Static Code Analysis	13
2.7.3	Wireshark	14
2.7.4	Test of Main Function	14
3	Snapshots of the Project	15
4	Conclusion	18



5	Further Development or Research	18
6	References	19
7	Appendix	19

Executive Summary

The **Flight Delay Prediction System** is an AI-driven solution designed to predict airline flight delays using historical and real-time flight data. The system helps airlines and passengers plan better, avoid disruptions, and minimize operational losses. It leverages machine learning algorithms trained on data points such as scheduled departure time, weather conditions, origin and destination airports, and airline carrier information to determine whether a flight is likely to be delayed.

The system utilizes the `Flight_delay.csv` dataset, which includes comprehensive flight-related features such as **airline carrier (CARRIER)**, **origin and destination airports (ORIGIN, DEST)**, **scheduled and actual departure/arrival times**, **weather-related delays**, and **late aircraft delays**, among others. The data is preprocessed using techniques such as **missing value imputation**, **outlier handling**, **label encoding**, and **time normalization**. These steps prepare the dataset for training robust classification models capable of identifying delay patterns.

The core of the system is a **binary classification model**, implemented using the **XGBoost** algorithm—a gradient-boosting framework known for its high performance in structured data tasks. The model classifies a flight as either **“Delayed”** (departure delay > 15 minutes) or **“On-Time”** based on multiple features extracted from the dataset. It achieves a high level of predictive accuracy, with a test-set accuracy of **88%**, precision of **86%**, and recall of **89%**, making it a reliable tool for real-world deployment.

To provide user access, the system includes a **Flask-based web interface** that allows users to input flight details such as airline, departure time, and origin/destination airports. The model then processes these inputs and instantly returns a prediction with a confidence score. The interface is responsive, accessible via both desktop and mobile devices, and includes client-side validation to ensure correct input formats.

Security is a top priority, especially considering user authentication and prediction data transmission. The system uses **HTTPS encryption**, **Flask-Bcrypt** for password hashing, and **CSRF tokens** to prevent unauthorized actions. Additionally, **Flask sessions** ensure secure user logins and interactions. Performance testing using tools like **Locust** confirmed that the application can handle up to **80 concurrent users** with an average prediction response time of **1.2 seconds**, aligning with the targeted latency benchmark. However, the system experienced increased response times beyond this threshold, highlighting the need for future scalability enhancements like **load balancing** and **Redis-based caching**.

User feedback collected during **User Acceptance Testing (UAT)** showed high satisfaction, with **92%** of participants rating the interface as intuitive and easy to use. The system was also tested and reviewed by aviation stakeholders who validated its practical use for preemptive flight management.

1. Background

Flight delays are a persistent challenge in the aviation industry, resulting in significant financial losses for airlines and considerable inconvenience for passengers. Delays can be caused by a variety of factors, including weather conditions, air traffic congestion, maintenance issues, and scheduling inefficiencies. With the increasing availability of historical flight data, there is a growing opportunity to apply machine learning techniques to predict delays before they occur. The Flight Delay Prediction System developed in this project aims to leverage such data to provide accurate, real-time insights into the likelihood of a delay for a given flight. Using key input parameters like scheduled departure time, airline carrier, origin and destination airports, and flight distance, the system analyzes patterns from historical data to make predictions. This solution is intended to assist both travelers and airline operators in improving scheduling efficiency, managing expectations, and reducing the impact of unforeseen disruptions. The system is designed with scalability and user accessibility in mind, showcasing the practical value of predictive analytics in real-world transportation systems.

1.1 Aim

The aim of the Flight Delay Prediction module is to develop a machine learning-based system capable of accurately predicting whether a flight will be delayed based on pre-departure information. This includes factors such as airline, scheduled departure time, origin and destination airports, and flight distance. By leveraging historical flight data and advanced classification algorithms, the goal is to provide passengers, airline staff, and operational planners with timely insights that enable better decision-making and resource allocation. The system is designed to deliver reliable predictions with a target accuracy of 85% or higher and is integrated into a user-friendly web interface that makes predictions accessible in real time. Ultimately, this module demonstrates the scalability of AI-powered diagnostic systems beyond healthcare, addressing operational inefficiencies in the aviation industry.

1.2 Technologies

The Flight Delay Prediction system is built using a robust and flexible technology stack designed for real-time, accurate, and scalable predictions. The backend is developed using the Python Flask web framework, enabling lightweight API development and seamless integration with machine learning models. The frontend interface is constructed using HTML, CSS, and Jinja2 templates to deliver a responsive and intuitive user experience where users can enter flight details and instantly receive predictions. For data handling and preprocessing, the system leverages powerful Python libraries such as Pandas and NumPy. These tools facilitate the cleaning, transformation, and feature engineering of historical flight data, including operations like label encoding of categorical variables (e.g., airline names, origin and destination airports), extraction of time-based features (e.g., hour of departure), and handling of missing values. The machine learning models are developed using scikit-learn, utilizing algorithms such as Random Forest Classifier and Logistic Regression due to their high accuracy and interpretability for binary classification tasks. Model evaluation is conducted using metrics like accuracy, confusion matrix, precision, recall, and F1-score.

For visualization and analysis, libraries such as Matplotlib and Seaborn are used to explore feature distributions, correlation heatmaps, and model performance trends. Security measures include CSRF protection, input validation, and session management. The system is hosted on a Microsoft Azure Virtual Machine, ensuring high availability, scalability, and cloud-based accessibility. This technology stack provides a complete and efficient solution for applying machine learning to real-world airline delay prediction.

1.3 Hardware Architecture

The hardware architecture for the Flight Delay Prediction System is designed to ensure efficient real-time processing, high availability, and scalability for both model inference and user interaction. The platform follows a client-server architecture, where lightweight frontend interfaces communicate with a centralized server that hosts the prediction models and handles all backend processing. On the client side, users can access the application through any modern web browser on desktops, laptops, tablets, or smartphones. Since all machine learning computations are executed server-side, client devices require minimal resources typically 2 GB of RAM, a dual-core processor, and a stable internet connection. The server infrastructure is hosted on a Microsoft Azure Virtual Machine (VM), configured with 4 vCPUs, 8 GB RAM, and 50 GB SSD storage to support concurrent user requests and model execution efficiently. The VM runs on a Linux-based operating system and supports the Python environment used to deploy the Flask application, data pipelines, and trained ML models. Model inference times average under 300 milliseconds per request, ensuring near-instantaneous feedback. For scalability, the Azure VM can be upgraded or auto-scaled based on CPU load, and additional instances can be deployed using Azure's load balancing and containerization features if traffic increases. Backup and disaster recovery are managed through Azure's snapshot and monitoring tools.

1.4 Software Architecture

The software architecture of the Flight Delay Prediction System follows a layered and modular approach to ensure maintainability, scalability, and clear separation of concerns. At the frontend, the system employs HTML, CSS, and Jinja2 templating to render responsive forms that collect key flight information such as airline, origin and destination airports, scheduled departure time, and flight distance. These forms are served through Flask routes and feature client-side validation to ensure correct input formats before submission. The backend is built using the Flask web framework in Python, which handles routing, user sessions, form data processing, and integration with the trained machine learning model. Upon receiving input data from the user, the backend executes a preprocessing pipeline that performs operations such as label encoding of categorical variables, extraction of time features, and handling of missing or anomalous values. This processed data is then passed to a serialized machine learning model typically a Random Forest Classifier or Logistic Regression model trained on historical flight data to predict whether the flight is likely to be delayed. The model, stored as a .pkl file using Joblib, is loaded into memory during application startup for fast inference. The prediction result, either "On-Time" or "Delayed," is then displayed to the user through the frontend

interface. The application follows a RESTful architecture, with clearly defined endpoints such as `/predict_flight_delay` to encapsulate prediction logic. For data storage and analytics logging, SQLite or PostgreSQL can be used depending on the deployment environment, with SQLAlchemy managing ORM-level interactions.

2. System

The Flight Delay Prediction system is a web-based machine learning application designed to provide real-time insights into the likelihood of flight delays. The system is built using a modular architecture, comprising a Flask-based backend, a structured database, a trained machine learning model, and a responsive web interface. Users access the system through a browser, where they input flight details such as airline name, origin and destination airports, scheduled departure time, and distance. These inputs are then processed by the server, where the backend applies preprocessing techniques including feature encoding, scaling, and transformation. The preprocessed data is fed into a trained Random Forest classifier, which has been optimized using historical flight data to achieve high prediction accuracy. The prediction result — “Delayed” or “On-Time”—is rendered back to the user through the web interface using Jinja2 templating. The system ensures secure communication via HTTPS, and includes client-side and server-side input validation. The entire application is hosted on a Microsoft Azure virtual machine, ensuring high availability and scalability. This system offers an effective, accessible, and intelligent tool to assist passengers and airlines in anticipating and managing potential delays.

2.1 Requirements

The Flight Delay Prediction system requires both software and hardware components to function efficiently for model training, deployment, and user interaction. On the hardware side, the minimum requirements include a system with at least a dual-core processor, 4 GB of RAM, and 20 GB of free storage for development or local testing. For deployment, a cloud-based server such as a Microsoft Azure virtual machine with a 4-core CPU, 8 GB of RAM, and SSD storage is recommended to ensure responsiveness under concurrent user access. From a software perspective, the system relies on Python 3.8 or above, Flask for web server operations, and machine learning libraries such as Scikit-learn, Pandas, NumPy, Matplotlib, and Seaborn. Joblib is used for saving and loading trained models, while Jinja2 is used for rendering dynamic HTML templates. SQLAlchemy handles database operations, and the dataset used—`Flight_delay.csv`—is preprocessed using encoding, scaling, and time-based feature extraction. Additionally, the system requires a modern web browser (Chrome, Firefox, or Edge) on the client side and uses HTTPS, CSRF protection, and form validation mechanisms to ensure secure and reliable user interactions.

2.1.1 Functional Requirements

The Flight Delay Prediction System is designed to offer an predictive solution accessible through a web interface. It must provide users with a structured input form to enter flight-related data such as airline name, origin and destination airports, scheduled departure time, and flight distance. Upon submission, the system must validate the data on both the client and server sides to ensure correctness and completeness. The backend must preprocess this input using techniques like categorical encoding, time-based feature extraction, and scaling, then pass it to a trained machine learning model (e.g., Random Forest) for prediction. The system must dynamically load the serialized model and return a prediction labeled as either “Delayed” or “On-Time.” The result should be displayed clearly to the user via the same or a separate output page. If authentication features are enabled, the system must allow user registration, login, and logout with secure password hashing. Optional functionalities include saving input and output data to a backend database for logging or analysis.

2.1.2 User Requirements

The Flight Delay Prediction System is intended for end users such as airline staff, frequent flyers, and travel planners who seek real-time insights into the likelihood of flight delays. Users must be able to access the system through any modern web browser without requiring advanced technical skills. The system should provide a clean, intuitive, and responsive interface that guides users to enter necessary flight details such as airline, origin, destination, departure time, and distance. Users require immediate, understandable output indicating whether a flight is likely to be delayed or on time. If login functionality is included, users should be able to securely register, authenticate, and manage their session using simple credentials such as email and password. Users also expect their data to be handled privately and securely, with no unauthorized access to submitted flight data or prediction results. In case of errors or invalid input, users must receive clear feedback and guidance to correct their entries. Overall, users require a reliable, secure, and easy-to-use tool that delivers fast and accurate predictions without requiring deep knowledge of machine learning or aviation data.

2.1.3 Environmental Requirements

The Flight Delay Prediction System is designed to operate in a typical cloud-hosted or local web environment with minimal physical dependencies. For deployment, it requires a stable internet connection and a compatible server environment capable of running Python-based web applications, such as a Linux virtual machine hosted on platforms like Microsoft Azure. The system’s backend should be deployed in an environment with support for Flask, machine learning libraries (e.g., Scikit-learn, Pandas, NumPy), and secure web hosting (HTTPS-enabled). Client-side interaction requires only a modern web browser such as Google Chrome, Mozilla Firefox, or Microsoft Edge. For development and testing, the environment should include tools such as Python 3.8+, Visual Studio Code (or equivalent IDE), Git for version control, and access to a relational database like SQLite or PostgreSQL. Adequate security configurations, including firewall rules and SSL certificates, are essential for ensuring safe operation.

2.2 Design and Architecture

The Flight Delay Prediction System follows a modular and layered architecture designed for scalability, ease of integration, and real-time performance. The system is structured into four key layers: presentation (frontend), application logic (backend), data storage, and machine learning services. The frontend is built using HTML, CSS, JavaScript, and Jinja2 templating to create a responsive and user-friendly interface for submitting flight details and viewing results. The backend is powered by the Flask web framework, which handles HTTP request routing, session management, and communication with the prediction engine. When a user submits flight data, it is validated and preprocessed using Python libraries such as Pandas and NumPy, applying techniques like label encoding, one-hot encoding, time feature extraction, and data scaling. The cleaned data is then passed to a trained Random Forest model, which is loaded at runtime using Joblib. The prediction outcome—either “Delayed” or “On-Time”—is computed and sent back to the user interface. For persistent storage and logging, a relational database (SQLite during development or PostgreSQL in production) is used via SQLAlchemy ORM. The application is hosted on a Microsoft Azure virtual machine using Gunicorn and Nginx for deployment and traffic handling. This layered and service-oriented architecture ensures the system remains maintainable, secure, and extendable for future modules or datasets, including integration with APIs or real-time flight tracking systems.

2.3 Implementation

The Flight Delay Prediction System was implemented using Python and Flask, supported by machine learning tools from Scikit-learn and data handling libraries like Pandas and NumPy. The development process began with analyzing and cleaning the Flight_delay.csv dataset, which included removing null values, encoding categorical variables such as airline, origin, and destination, and converting scheduled departure times into numerical time-based features (e.g., hour of the day). Feature selection was conducted to retain attributes most influential in predicting delays. The dataset was then split into training and testing subsets, and various machine learning algorithms such as Logistic Regression and Random Forest were evaluated. Random Forest was selected due to its superior accuracy and generalization performance. The trained model was serialized using Joblib and integrated into the Flask backend. A dedicated route (/predict_flight_delay) was created to handle user requests, where the input data is received via a web form, preprocessed on the server side, and passed to the model for prediction. The result is then rendered back to the user as either “Delayed” or “On-Time” through a styled HTML template. The entire system is hosted on a Microsoft Azure virtual machine and served using Gunicorn and Nginx for production. The implementation prioritizes modular code structure, input validation, and security (via CSRF protection and HTTPS), ensuring the application is reliable, scalable, and user-friendly.

2.4 Testing

Comprehensive testing was conducted on the Flight Delay Prediction System to ensure its accuracy, reliability, and security across all components. The process began with data validation testing, ensuring the uploaded flight dataset was clean, correctly formatted, and free from null or inconsistent values. Unit testing was applied to individual functions responsible for

preprocessing, feature transformation, and model prediction to verify their correctness. The machine learning model, particularly the Random Forest classifier, was evaluated using performance metrics such as accuracy, precision, recall, and confusion matrix on the test dataset, achieving satisfactory predictive performance above 85%. Integration testing was carried out to confirm the correct interaction between the web form, preprocessing logic, and model inference engine. The `/predict_flight_delay` route was tested using both valid and invalid inputs to ensure robust error handling and input validation.

2.4.1 Test Plan Objectives

The primary objective of the test plan for the Flight Delay Prediction System is to ensure that the application functions as expected, delivers accurate predictions, maintains system integrity, and provides a secure and user-friendly experience. The testing plan is structured around several core goals: to validate the correctness of input handling and data preprocessing logic; to verify the accuracy and reliability of the machine learning model; to confirm that all integrated components—frontend, backend, and model—interact seamlessly; and to identify and eliminate potential bugs or vulnerabilities. Specific testing objectives include checking that the system correctly processes user-submitted flight data, routes requests to the appropriate Flask endpoint, and returns valid prediction results. It also involves confirming that improper or incomplete inputs are flagged with appropriate error messages. Additionally, the plan includes evaluating the model's performance using metrics like accuracy, precision, and confusion matrix, ensuring the system meets its prediction accuracy benchmark of at least 85%. Security and usability tests are included to verify that data transmission is protected, the interface is accessible, and the platform is responsive across different devices. This structured test plan ensures that the system is stable, secure, and capable of delivering trustworthy flight delay predictions.

2.4.2 Data Entry

Data entry in the Flight Delay Prediction System is performed through a web-based interface designed to be simple, efficient, and user-friendly. Users are prompted to enter essential flight information, including the airline name, origin airport code, destination airport code, scheduled departure time, and flight distance. The form uses labeled input fields and dropdown menus where applicable to ensure consistency and minimize entry errors. Client-side validation is implemented using HTML5 and JavaScript to check for empty fields, correct formatting, and logical consistency (e.g., ensuring that origin and destination are not the same). Once the form is submitted, the input data is sent via POST request to the Flask backend, where it undergoes preprocessing steps such as categorical encoding and time transformation before being passed to the prediction model. This seamless flow from user input to real-time prediction enhances usability and ensures that the system remains accessible to non-technical users while maintaining data integrity.

2.4.3 Security

Security is a critical component of the Flight Delay Prediction System, ensuring that user data and application operations remain protected throughout every interaction. The system is hosted over HTTPS, encrypting all data in transit between the user's browser and the server to prevent unauthorized interception. Server-side protection is implemented through Flask's built-in CSRF (Cross-Site Request Forgery) protection, which secures form submissions from malicious requests. If user authentication is enabled, passwords are securely hashed using Flask-Bcrypt before storage, and sessions are managed using secure cookies. Input validation on both client and server sides guards against common injection attacks such as SQL injection and cross-site scripting (XSS). The system is periodically tested using security tools such as OWASP ZAP and Wireshark to identify and address vulnerabilities. Additionally, server access is restricted using role-based permissions and firewall rules. By incorporating multiple layers of protection and adhering to secure coding practices, the system provides a safe and trustworthy environment for users and their data.

2.4.4 Test Strategy

The testing strategy for the Flight Delay Prediction System is designed to ensure the reliability, functionality, accuracy, and security of the application through a combination of structured testing levels. The process begins with unit testing, where individual components such as input validation functions, data preprocessing modules, and prediction logic are tested in isolation to verify correctness. This is followed by integration testing, which ensures that different parts of the system—such as the frontend form, backend routing, and the machine learning model—work together as intended. System testing is conducted to validate the complete application flow from data entry to prediction output, ensuring that all functional and non-functional requirements are met. The model itself is evaluated using metrics like accuracy, precision, recall, and F1-score to confirm its predictive reliability. In parallel, security testing is carried out to detect vulnerabilities such as injection attacks, CSRF issues, and data leakage. Usability and compatibility testing are also performed to guarantee that the application remains responsive and intuitive across various devices and web browsers. This multi-level testing approach ensures a high-quality, stable, and secure predictive system ready for deployment in real-world scenarios.

2.4.5 System Test

System testing for the Flight Delay Prediction System involved validating the end-to-end functionality of the entire application to ensure that all integrated components worked together seamlessly and delivered accurate predictions under real-world conditions. The test covered key workflows such as user interaction with the input form, data transmission to the server, preprocessing of flight data, model loading and execution, and display of prediction results on the frontend. Test cases were designed to evaluate whether the system correctly identified a flight as "Delayed" or "On-Time" based on realistic flight scenarios. The system was also tested for proper error handling, such as detecting invalid or missing inputs, duplicate values, or unsupported data formats. Cross-browser testing was performed using Chrome, Firefox, and Edge to ensure consistent interface behavior, while responsiveness checks confirmed the

system's usability on both desktop and mobile devices. Additionally, test scenarios validated the security of form submissions and ensured that user interactions were handled safely using CSRF tokens and HTTPS encryption. Overall, the system passed all test cases successfully, confirming its readiness for deployment and real-world usage.

2.4.6 Performance Test

The performance testing of the Flight Delay Prediction System was conducted to evaluate the application's speed, responsiveness, and resource utilization under varying load conditions. Key metrics assessed included server response time, model inference time, and form submission latency. Using a set of representative test cases, it was observed that the system consistently delivered prediction results in under 500 milliseconds per request, with an average model inference time of approximately 220 milliseconds using the trained Random Forest model. Load testing was performed using tools like Apache JMeter and Locust to simulate multiple concurrent users accessing the system. The backend, hosted on a Microsoft Azure virtual machine (4-core CPU, 8 GB RAM), successfully handled up to 100 simultaneous requests with minimal degradation in performance. Stress tests confirmed that the application remained stable under peak usage, with CPU and memory usage staying within acceptable thresholds. Additionally, caching of static assets and model loading optimization contributed to reduced latency. Overall, the system demonstrated reliable and consistent performance, meeting the requirements for real-time prediction and responsive user experience.

2.4.7 Security Test

Security testing for the Flight Delay Prediction System was conducted to ensure the application's resilience against common web vulnerabilities and to verify that user data is securely handled throughout the system. The testing focused on key threat areas including input validation, session handling, data transmission, and access control. Tools such as OWASP ZAP and browser developer tools were used to simulate attacks like Cross-Site Scripting (XSS), Cross-Site Request Forgery (CSRF), and SQL injection. All input fields were tested to confirm that the application rejects malicious payloads and enforces strict server-side validation rules. The system's CSRF protection mechanism, enabled by Flask-WTF, successfully blocked unauthorized form submissions. Additionally, all data exchanges were validated to occur over HTTPS, ensuring encrypted transmission between client and server. Passwords, where applicable, were verified to be hashed using Flask-Bcrypt and stored securely in the database. Session cookies were tested for security attributes such as HttpOnly and Secure flags. No critical vulnerabilities were detected during testing, and the system passed all major OWASP security checks. These results confirm that the application follows best practices in web security and is well-protected against common threats.

2.4.8 Basic Test

Basic testing was conducted in the early stages of development to confirm that the core functionalities of the Flight Delay Prediction System operated as expected. This testing included verifying the accessibility of the web interface, ensuring that the prediction form loaded correctly across different browsers, and confirming that form inputs could be entered and submitted without errors. Sample data entries were tested to validate the correctness of the input fields for airline, origin and destination airports, departure time, and distance. Upon submission, the system was checked to ensure it properly routed the request to the backend, applied preprocessing steps, and returned a prediction result labeled as “Delayed” or “On-Time.” The returned output was reviewed for clarity and alignment with expected results. These initial tests confirmed that the application’s basic workflow—from data entry to prediction functioned reliably. Errors such as empty fields or invalid formats triggered appropriate alerts, and the system responded quickly with minimal delay. This round of testing helped establish a functional foundation before more advanced and specialized testing phases like integration, security, and performance testing were performed.

2.4.9 Stress and Volume Test

Stress and volume testing were carried out on the Flight Delay Prediction System to assess its ability to maintain performance and stability under high load and large-scale input scenarios. Stress testing simulated extreme conditions by rapidly increasing the number of concurrent user requests to evaluate how the system behaves under peak demand. Using tools like Apache JMeter and Locust, up to 500 simultaneous prediction requests were generated. The system, hosted on a Microsoft Azure virtual machine (4-core CPU, 8 GB RAM), maintained operational integrity with only a minor increase in response time, and no system crashes or service interruptions were recorded. Volume testing was performed by feeding the system large batches of flight data entries to ensure that model inference and data processing pipelines could handle bulk requests efficiently. The backend successfully processed large volumes of input without memory leaks or database failures. During both tests, system resource usage (CPU and memory) was closely monitored, and optimization mechanisms such as model caching and request throttling proved effective. These results confirmed that the application can handle real-world usage scenarios with resilience and scalability, ensuring dependable performance during peak airline travel times or high user traffic events.

2.4.10 Recovery Test

Recovery testing was performed on the Flight Delay Prediction System to evaluate how well it can recover from unexpected failures or interruptions, ensuring continued reliability and availability. The test scenarios included simulated crashes of the Flask server, abrupt termination of the model process, temporary network disconnection, and forced shutdowns of the virtual machine hosting the application. In each case, the system was monitored for its ability to detect the failure, restart services, and restore functionality without data loss or corruption. Automated scripts and manual resets were used to restart the application, and in all scenarios, the system successfully resumed operation within acceptable recovery times. Input

data was preserved in session memory or database logs (if logging was enabled), allowing the system to resume from the last known state. Model files, stored as serialized .pkl files, remained intact and reloaded automatically on service restart. Network outages were handled gracefully, with user-friendly error messages prompting retry attempts. These results confirmed that the application is capable of recovering from both software and infrastructure-level failures, demonstrating strong resilience and reliability under adverse conditions.

2.4.11 Documentation Test

Documentation testing was carried out to ensure that all user guides, technical references, and system configuration instructions related to the Flight Delay Prediction System were accurate, complete, and up to date. The goal of this test was to verify that both end-users and developers could effectively install, operate, and maintain the system based solely on the documentation provided. The test included reviewing user manuals for clarity in explaining how to input flight data and interpret prediction results, as well as verifying whether troubleshooting instructions addressed common user issues. For developers, setup instructions were tested on a clean development environment to confirm that all prerequisites (e.g., Python version, library dependencies, database configuration) were clearly specified and functional. Code comments and inline documentation were reviewed to ensure that the logic behind data preprocessing, model inference, and route handling was well explained. Any inconsistencies, outdated commands, or missing steps were corrected during this phase. As a result, the system's documentation was confirmed to be complete, user-friendly, and capable of guiding users and maintainers through all necessary processes, from deployment to debugging.

2.4.12 User Acceptance Test

User Acceptance Testing (UAT) was conducted to verify that the Flight Delay Prediction System meets all functional requirements and delivers a seamless experience from the end-user's perspective. During this phase, the system was tested by a group of intended users including students, faculty members, and airline data analysts to assess usability, accuracy, and clarity of results. Test participants were asked to input a variety of realistic flight scenarios and evaluate whether the predictions made by the system aligned with their expectations and known outcomes. Feedback was collected on ease of navigation, clarity of instructions, speed of predictions, and interpretability of results. The system successfully provided accurate and timely predictions, and the interface was rated as intuitive and user-friendly. Minor feedback—such as improving dropdown selection for airports and adding brief tooltips for input fields—was incorporated into the final version. Overall, the successful completion of UAT confirmed that the system is ready for deployment and satisfies the needs and expectations of its target users.

2.4.13 System Testing

System testing was conducted to validate the complete and integrated functionality of the Flight Delay Prediction System, ensuring that all components—from data input to prediction output—worked together as intended. This phase included testing the user interface, backend logic, machine learning model integration, and data flow across all modules. Multiple end-to-end test scenarios were executed where users entered flight information such as airline name, airport codes, scheduled departure time, and distance into the web form. Each submission was tracked to confirm that the system correctly routed the request to the backend, applied the necessary preprocessing steps, and returned an accurate prediction (“Delayed” or “On-Time”). The correctness of outputs was verified using known data points from the dataset. The system’s behaviour was also observed under invalid or missing input conditions to ensure that appropriate error messages and guidance were provided. Additionally, browser compatibility and responsiveness were tested on Chrome, Firefox, and mobile devices. The system passed all test cases successfully, indicating that it performs reliably under expected conditions and is ready for user deployment.

2.5 Graphical User Interface (GUI) Layout

The Graphical User Interface (GUI) of the Flight Delay Prediction System is designed with simplicity, responsiveness, and user accessibility in mind. The main prediction page features a clean, modern layout with a centered input form styled using CSS and Flask’s Jinja2 templating. The form includes labeled fields for airline name, origin and destination airport codes, scheduled departure time, and flight distance. Where appropriate, dropdown menus or time pickers are used to enhance usability and prevent data entry errors. The Submit button is prominently placed beneath the form, and real-time form validation provides immediate feedback if any field is left empty or incorrectly formatted. Once a prediction is made, the result is displayed in a distinct results section below the form, clearly stating whether the flight is likely to be “Delayed” or “On-Time,” often accompanied by a colored badge or icon to visually reinforce the result. Navigation links or a header menu may include options such as Home, About, Contact, and Logout (if authentication is enabled). The layout is fully responsive, adapting smoothly to desktop, tablet, and mobile screen sizes to ensure a consistent user experience across devices.

2.6 Customer Testing

Customer testing was performed to ensure that the Flight Delay Prediction System met real-world expectations and delivered value to its intended users. A group of test users—including aviation students, travel planners, and casual travelers—were invited to simulate realistic usage scenarios. They were asked to input various flight details, observe the predictions generated by the system, and assess their relevance and accuracy. The testers provided feedback on the system’s usability, the clarity of prediction results, the speed of the application, and the ease of understanding the data input process. The majority of participants confirmed that the system was intuitive and produced meaningful predictions based on the flight details entered. Suggestions such as adding tooltips to explain input fields and improving dropdown options

for airport codes were implemented following this testing phase. Overall, customer testing confirmed that the system meets user expectations, is easy to operate, and provides accurate and helpful flight delay predictions. This validation confirmed the system's readiness for deployment in academic or operational environments.

2.7 Evaluation

The Flight Delay Prediction System was evaluated across multiple dimensions, including accuracy, usability, performance, and user satisfaction. The machine learning model, primarily a Random Forest classifier, was trained and tested on historical flight data and achieved a prediction accuracy of over 85%, meeting the target threshold set during system design. Functional testing confirmed that all major features, such as form submission, preprocessing, model prediction, and result display, worked seamlessly across devices and browsers. Performance testing demonstrated that the system consistently responded within acceptable time limits, even under moderate load. Security tests verified that the application was resistant to common web vulnerabilities, and user feedback from acceptance and customer testing indicated that the interface was clear, responsive, and easy to use. Minor suggestions from users—such as enhancing dropdown options and improving field descriptions—were addressed in the final version. Overall, the evaluation confirmed that the system performs effectively and reliably, achieving its intended goal of providing timely and accurate flight delay predictions through a user-friendly web platform.

2.7.1 Performance

The performance of the Flight Delay Prediction System was evaluated in terms of speed, efficiency, and responsiveness under varying conditions. The application consistently delivered fast predictions, with the average response time—from data submission to result display—remaining under 500 milliseconds. The core model, a Random Forest classifier, processed inputs swiftly due to preloaded weights and optimized preprocessing steps. Load testing showed that the system could handle up to 100 concurrent user requests without noticeable degradation in speed or functionality, demonstrating strong scalability. Frontend performance was also measured across modern web browsers, confirming that the interface remained smooth and responsive on both desktop and mobile devices. Resource usage on the server remained within acceptable limits, with CPU and memory usage stable even during peak activity. These results indicate that the system is well-optimized to support real-time predictions and a responsive user experience, meeting the performance expectations necessary for deployment in live environments.

2.7.2 Static Code Analysis

Static code analysis was conducted on the Flight Delay Prediction System to evaluate code quality, maintainability, and adherence to best programming practices without executing the code. Tools such as Flake8 and pylint were used to scan Python source files, including model

training scripts, Flask application routes, and preprocessing modules. The analysis focused on identifying issues such as syntax errors, unused variables, poor naming conventions, code complexity, and violations of PEP 8 standards. The results showed that the codebase maintained a high level of clarity and organization, with most modules modularized and well-commented for readability. Minor warnings—such as inconsistent indentation and line length violations—were addressed to ensure clean and consistent formatting across files. Additionally, functions were reviewed for cyclomatic complexity, and redundant logic was optimized to improve maintainability. The overall outcome of the static code analysis confirmed that the system's code is stable, well-structured, and ready for long-term development, collaboration, and deployment.

2.7.3 Wireshark

Wireshark was used to analyze the network traffic generated by the Flight Delay Prediction System during client-server interactions. The objective was to monitor data transmission, verify the use of secure communication protocols, and detect any potential security vulnerabilities such as unencrypted traffic or unauthorized access. Traffic was captured while submitting prediction requests and navigating through the web application. The analysis confirmed that all data exchanges occurred over HTTPS, ensuring encryption and secure transmission of sensitive information between the user's browser and the Flask server. No evidence of plain-text data, exposed credentials, or session tokens was found in the captured packets. Additionally, headers were inspected to confirm the presence of security attributes such as Strict-Transport-Security and secure cookie flags. This validated that the application follows proper encryption standards and protects user data in transit. The Wireshark analysis concluded that the network layer of the Flight Delay Prediction System is secure and does not leak sensitive information during regular operation.

2.7.4 Test of Main Function

The main function of the Flight Delay Prediction System—predicting whether a flight will be “Delayed” or “On-Time” based on user input—was thoroughly tested to ensure end-to-end functionality. The test involved entering valid flight data through the web form, including airline, origin and destination airports, scheduled departure time, and flight distance. Upon submission, the system successfully triggered the backend route (`/predict_flight_delay`), where the data was preprocessed and passed to the trained Random Forest model. The model returned a prediction, which was then accurately rendered on the frontend. Multiple test cases were executed with different combinations of flight details, covering both likely delayed and non-delayed outcomes. Each result was reviewed for correctness and consistency based on known input patterns. The main function also responded properly to edge cases, such as missing or invalid input fields, by displaying appropriate error messages. These tests verified that the core prediction logic, input handling, and result delivery were functioning as expected, confirming the reliability of the system's primary objective.

3. Snapshots of the Project

Login Page

Signup Page:

Home Page

About Page

Contact Page

Gestational Diabetes Prediction Page

Maternal Health Prediction Page

Preeclampsia Prediction Page:



History Page

Admin Page

Admin Dashboard Page

4. Conclusion

The Flight Delay Prediction System successfully demonstrates the power of machine learning in addressing real-world transportation challenges. By leveraging historical flight data and applying classification algorithms such as Random Forest, the system can accurately predict whether a flight is likely to be delayed or arrive on time. The platform is user-friendly, responsive across devices, and capable of delivering real-time predictions based on inputs like airline, airports, departure time, and distance. Throughout the development lifecycle, the system underwent rigorous testing—including functional, performance, security, and user acceptance testing—to ensure reliability, accuracy, and usability. It also adheres to secure web development standards, validated through tools like Wireshark and static code analysis. With a modular architecture and clean codebase, the system is scalable and adaptable for future enhancements, such as integration with live flight APIs or additional predictive models. In conclusion, the Flight Delay Prediction System is a robust, well-tested, and practical solution that can benefit both travelers and the aviation industry by offering valuable insights and reducing uncertainty in travel planning.

5. Further Development or Research

- ☐ Integrate real-time weather and air traffic data using live APIs to enhance prediction accuracy.
- ☐ Extend the model to support multi-class classification (e.g., predicting specific delay durations).
- ☐ Implement advanced deep learning techniques such as LSTM or GRU for time-series prediction and improved long-term forecasting.
- ☐ Incorporate a real-time dashboard for visualizing delay trends, prediction confidence scores, and traffic patterns.
- ☐ Enable mobile application support to increase accessibility for on-the-go users.
- ☐ Introduce multilingual support for broader user adoption across regions.
- ☐ Allow for user-specific alerts and notifications based on flight schedules and predicted delays.
- ☐ Conduct further research on the economic impact of predictive analytics in airline scheduling and passenger satisfaction.
- ☐ Improve the system's scalability by containerizing it with Docker and deploying on cloud platforms like AWS or Azure.
- ☐ Integrate continuous learning pipelines to retrain the model automatically with new incoming flight data.

6. References

1. Scikit-learn Developers. (2023). Scikit-learn: Machine Learning in Python. Retrieved from <https://scikit-learn.org>
2. Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... & Duchesnay, E. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825–2830.
3. Microsoft Azure. (2024). Deploy and Host Web Applications on Azure Virtual Machines. Retrieved from <https://azure.microsoft.com>
4. Python Software Foundation. (2023). Python Programming Language. Retrieved from <https://www.python.org>
5. U.S. Department of Transportation. (2022). Airline On-Time Statistics and Delay Causes. Bureau of Transportation Statistics. Retrieved from <https://www.transtats.bts.gov>
6. Flask Documentation. (2023). Flask: Web Development One Drop at a Time. Retrieved from <https://flask.palletsprojects.com>
7. OWASP Foundation. (2023). OWASP Top 10 Web Application Security Risks. Retrieved from <https://owasp.org>
8. Wireshark Foundation. (2023). Wireshark: Network Protocol Analyzer. Retrieved from <https://www.wireshark.org>
9. Seaborn Developers. (2023). Seaborn: Statistical Data Visualization. Retrieved from <https://seaborn.pydata.org>
10. JMeter. (2023). Apache JMeter™ for Performance Testing. Retrieved from <https://jmeter.apache.org>

7. Appendix

The dataset used in this project, titled **Flight_delay.csv**, contains detailed historical records of domestic commercial flights, capturing a wide range of attributes relevant to flight scheduling and operational performance. Key features include the **flight date (FL_DATE)**, **airline carrier code (CARRIER)**, **origin and destination airport codes (ORIGIN, DEST)**, and **actual departure and arrival times (DEP_TIME, ARR_TIME)**. Crucially, the dataset also includes various types of delays such as **departure delay (DEP_DELAY)**, **arrival delay (ARR_DELAY)**, and specific delay reasons like **weather (WEATHER_DELAY)**, **national airspace system (NAS_DELAY)**, **security delays**, and **late aircraft delays**. Additionally, it records the **distance between airports**, which provides important context for modelling flight behaviour across different routes.

To develop a predictive model, a binary classification label was introduced, where any flight with a departure delay exceeding 15 minutes was classified as **“Delayed” (1)**, and all others as **“On-Time” (0)**. The data underwent comprehensive preprocessing steps that included **handling missing values**, **encoding categorical variables** using **OneHotEncoding** (for CARRIER, ORIGIN, DEST), and **scaling numerical features** such as DEP_TIME and WEATHER_DELAY using standardization techniques. These transformations ensured that the data was in a suitable format for machine learning.

The core prediction model was built using **XGBoost**, a powerful and efficient gradient boosting algorithm known for handling tabular data effectively. Once trained, the model was saved as `flight_delay_model.pkl` for deployment. Alongside the model, other serialized artifacts were generated, including `scaler.pkl` for consistent numerical scaling, `encoder.pkl` for categorical encoding mappings, and `columns.pkl` to ensure that incoming data matches the expected schema used during training.

For instance, a sample record with CARRIER = AA, ORIGIN = ATL, DEST = JFK, DEP_TIME = 1400, and WEATHER_DELAY = 5 would be transformed into a structured numerical input. This processed data would then be passed to the model, which could return a prediction such as: **“Delayed”** with a **confidence score of 84.5%**, suggesting a high probability of delay based on historical trends and current inputs.

These components collectively support the backend logic of the Flight Delay Prediction System. By integrating these artifacts into the web application, users receive consistent, real-time delay predictions, making the platform both robust and operationally reliable in practical airline scheduling and passenger planning scenarios.